Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter I.3 (by J.K. Reid).[2]

George, A., and Liu, J.W.H. 1981, *Computer Solution of Large Sparse Positive Definite Systems* (Englewood Cliffs, NJ: Prentice-Hall).[3]

*NAG Fortran Library* (Oxford, UK: Numerical Algorithms Group), see 2007+, `http://www.nag.co.uk`.[4]

*IMSL Math/Library Users Manual* (Houston: IMSL Inc.), see 2007+, `http://www.vni.com/products/imsl`.[5]

Eisenstat, S.C., Gursky, M.C., Schultz, M.H., and Sherman, A.H. 1977, *Yale Sparse Matrix Package*, Technical Reports 112 and 114 (Yale University Department of Computer Science).[6]

Bai, Z., Demmel, J., Dongarra, J. Ruhe, A., and van der Vorst, H. (eds.) 2000, *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide* Ch. 10 (Philadelphia: S.I.A.M.). Online at URL in `http://www.cs.ucdavis.edu/~bai/ET/contents.html`.[7]

*SPARSKIT*, 2007+, at `http://www-users.cs.umn.edu/~saad/software/SPARSKIT/sparskit.html`.[8]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapters 4 and 10, particularly §10.2–§10.3.[9]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 8.[10]

Baker, L. 1991, *More C Tools for Scientists and Engineers* (New York: McGraw-Hill).[11]

Fletcher, R. 1976, in *Numerical Analysis Dundee 1975*, Lecture Notes in Mathematics, vol. 506, A. Dold and B Eckmann, eds. (Berlin: Springer), pp. 73–89.[12]

*PCGPAK User's Guide* (New Haven: Scientific Computing Associates, Inc.).[13]

Saad, Y., and Schulz, M. 1986, *SIAM Journal on Scientific and Statistical Computing*, vol. 7, pp. 856–869.[14]

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.

Bunch, J.R., and Rose, D.J. (eds.) 1976, *Sparse Matrix Computations* (New York: Academic Press).

Duff, I.S., and Stewart, G.W. (eds.) 1979, *Sparse Matrix Proceedings 1978* (Philadelphia: S.I.A.M.).

## 2.8 Vandermonde Matrices and Toeplitz Matrices

In §2.4 the case of a tridiagonal matrix was treated specially, because that particular type of linear system admits a solution in only of order $N$ operations, rather than of order $N^3$ for the general linear problem. When such particular types exist, it is important to know about them. Your computational savings, should you ever happen to be working on a problem that involves the right kind of particular type, can be enormous.

This section treats two special types of matrices that can be solved in of order $N^2$ operations, not as good as tridiagonal, but a lot better than the general case. (Other than the operations count, these two types having nothing in common.) Matrices of the first type, termed *Vandermonde matrices*, occur in some problems having to do with the fitting of polynomials, the reconstruction of distributions from their moments, and also other contexts. In this book, for example, a Vandermonde problem crops up in §3.5. Matrices of the second type, termed *Toeplitz matrices*, tend

to occur in problems involving deconvolution and signal processing. In this book, a Toeplitz problem is encountered in §13.7.

These are not the only special types of matrices worth knowing about. The *Hilbert matrices*, whose components are of the form $a_{ij} = 1/(i + j + 1)$, $i, j = 0, \ldots, N-1$, can be inverted by an exact integer algorithm and are very difficult to invert in any other way, since they are notoriously ill-conditioned (see [1] for details). The Sherman-Morrison and Woodbury formulas, discussed in §2.7, can sometimes be used to convert new special forms into old ones. Reference [2] gives some other special forms. We have not found these additional forms to arise as frequently as the two that we now discuss.

### 2.8.1 Vandermonde Matrices

A Vandermonde matrix of size $N \times N$ is completely determined by $N$ arbitrary numbers $x_0, x_1, \ldots, x_{N-1}$, in terms of which its $N^2$ components are the integer powers $x_i^j$, $i, j = 0, \ldots, N-1$. Evidently there are two possible such forms, depending on whether we view the $i$'s as rows and $j$'s as columns, or vice versa. In the former case, we get a linear system of equations that looks like this,

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{2.8.1}$$

Performing the matrix multiplication, you will see that this equation solves for the unknown coefficients $c_i$ that fit a polynomial to the $N$ pairs of abscissas and ordinates $(x_j, y_j)$. Precisely this problem will arise in §3.5, and the routine given there will solve (2.8.1) by the method that we are about to describe.

The alternative identification of rows and columns leads to the set of equations

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_{N-1} \\ x_0^2 & x_1^2 & \cdots & x_{N-1}^2 \\ & & \cdots & \\ x_0^{N-1} & x_1^{N-1} & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \cdots \\ w_{N-1} \end{bmatrix} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ \cdots \\ q_{N-1} \end{bmatrix} \tag{2.8.2}$$

Write this out and you will see that it relates to the *problem of moments*: Given the values of $N$ points $x_i$, find the unknown weights $w_i$, assigned so as to match the given values $q_j$ of the first $N$ moments. (For more on this problem, consult [3].) The routine given in this section solves (2.8.2).

The method of solution of both (2.8.1) and (2.8.2) is closely related to Lagrange's polynomial interpolation formula, which we will not formally meet until §3.2. Notwithstanding, the following derivation should be comprehensible:

Let $P_j(x)$ be the polynomial of degree $N-1$ defined by

$$P_j(x) = \prod_{\substack{n=0 \\ n \neq j}}^{N-1} \frac{x - x_n}{x_j - x_n} = \sum_{k=0}^{N-1} A_{jk} x^k \tag{2.8.3}$$

Here the meaning of the last equality is to define the components of the matrix $A_{ij}$ as the coefficients that arise when the product is multiplied out and like terms collected.

The polynomial $P_j(x)$ is a function of $x$ generally. But you will notice that it is specifically designed so that it takes on a value of zero at all $x_i$ with $i \neq j$ and has a value of unity at $x = x_j$. In other words,

$$P_j(x_i) = \delta_{ij} = \sum_{k=0}^{N-1} A_{jk} x_i^k \tag{2.8.4}$$

But (2.8.4) says that $A_{jk}$ is exactly the inverse of the matrix of components $x_i^k$, which appears in (2.8.2), with the subscript as the column index. Therefore the solution of (2.8.2) is just that matrix inverse times the right-hand side,

$$w_j = \sum_{k=0}^{N-1} A_{jk} q_k \tag{2.8.5}$$

As for the transpose problem (2.8.1), we can use the fact that the inverse of the transpose is the transpose of the inverse, so

$$c_j = \sum_{k=0}^{N-1} A_{kj} y_k \tag{2.8.6}$$

The routine in §3.5 implements this.

It remains to find a good way of multiplying out the monomial terms in (2.8.3), in order to get the components of $A_{jk}$. This is essentially a bookkeeping problem, and we will let you read the routine itself to see how it can be solved. One trick is to define a master $P(x)$ by

$$P(x) \equiv \prod_{n=0}^{N-1} (x - x_n) \tag{2.8.7}$$

work out its coefficients, and then obtain the numerators and denominators of the specific $P_j$'s via synthetic division by the one supernumerary term. (See §5.1 for more on synthetic division.) Since each such division is only a process of order $N$, the total procedure is of order $N^2$.

You should be warned that Vandermonde systems are notoriously ill-conditioned, by their very nature. (As an aside anticipating §5.8, the reason is the same as that which makes Chebyshev fitting so impressively accurate: There exist high-order polynomials that are very good uniform fits to zero. Hence roundoff error can introduce rather substantial coefficients of the leading terms of these polynomials.) It is a good idea always to compute Vandermonde problems in double precision or higher.

The routine for (2.8.2) that follows is due to G.B. Rybicki.

vander.h

```
void vander(VecDoub_I &x, VecDoub_O &w, VecDoub_I &q)
```

Solves the Vandermonde linear system $\sum_{i=0}^{N-1} x_i^k w_i = q_k$ $(k = 0, \ldots, N-1)$. Input consists of the vectors `x[0..n-1]` and `q[0..n-1]`; the vector `w[0..n-1]` is output.

```
{
    Int i,j,k,n=q.size();
    Doub b,s,t,xx;
    VecDoub c(n);
    if (n == 1) w[0]=q[0];
    else {
        for (i=0;i<n;i++) c[i]=0.0;          Initialize array.
        c[n-1] = -x[0];                      Coefficients of the master polynomial are found
        for (i=1;i<n;i++) {                      by recursion.
            xx = -x[i];
            for (j=(n-1-i);j<(n-1);j++) c[j] += xx*c[j+1];
            c[n-1] += xx;
        }
        for (i=0;i<n;i++) {                  Each subfactor in turn
```

```
            xx=x[i];
            t=b=1.0;
            s=q[n-1];
            for (k=n-1;k>0;k--) {          is synthetically divided,
                b=c[k]+xx*b;
                s += q[k-1]*b;             matrix-multiplied by the right-hand side,
                t=xx*t+b;
            }
            w[i]=s/t;                      and supplied with a denominator.
        }
    }
}
```

### 2.8.2 Toeplitz Matrices

An $N \times N$ Toeplitz matrix is specified by giving $2N-1$ numbers $R_k$, where the index $k$ ranges over $k = -N+1, \ldots, -1, 0, 1, \ldots, N-1$. Those numbers are then emplaced as matrix elements constant along the (upper-left to lower-right) diagonals of the matrix:

$$\begin{bmatrix} R_0 & R_{-1} & R_{-2} & \cdots & R_{-(N-2)} & R_{-(N-1)} \\ R_1 & R_0 & R_{-1} & \cdots & R_{-(N-3)} & R_{-(N-2)} \\ R_2 & R_1 & R_0 & \cdots & R_{-(N-4)} & R_{-(N-3)} \\ \cdots & & & \cdots & & \\ R_{N-2} & R_{N-3} & R_{N-4} & \cdots & R_0 & R_{-1} \\ R_{N-1} & R_{N-2} & R_{N-3} & \cdots & R_1 & R_0 \end{bmatrix} \tag{2.8.8}$$

The linear Toeplitz problem can thus be written as

$$\sum_{j=0}^{N-1} R_{i-j} x_j = y_i \qquad (i = 0, \ldots, N-1) \tag{2.8.9}$$

where the $x_j$'s, $j = 0, \ldots, N-1$, are the unknowns to be solved for.

The Toeplitz matrix is symmetric if $R_k = R_{-k}$ for all $k$. Levinson [4] developed an algorithm for fast solution of the symmetric Toeplitz problem, by a *bordering method*, that is, a recursive procedure that solves the $(M+1)$-dimensional Toeplitz problem

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad (i = 0, \ldots, M) \tag{2.8.10}$$

in turn for $M = 0, 1, \ldots$ until $M = N-1$, the desired result, is finally reached. The vector $x_j^{(M)}$ is the result at the $M$th stage and becomes the desired answer only when $N-1$ is reached.

Levinson's method is well documented in standard texts (e.g., [5]). The useful fact that the method generalizes to the *nonsymmetric* case seems to be less well known. At some risk of excessive detail, we therefore give a derivation here, due to G.B. Rybicki.

In following a recursion from step $M$ to step $M+1$ we find that our developing solution $x^{(M)}$ changes in this way:

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.11}$$

becomes

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M+1)} + R_{i-(M+1)} x_{M+1}^{(M+1)} = y_i \qquad i = 0, \ldots, M+1 \tag{2.8.12}$$

By eliminating $y_i$ we find

$$\sum_{j=0}^{M} R_{i-j}\left(\frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}}\right) = R_{i-(M+1)} \qquad i = 0, \ldots, M \tag{2.8.13}$$

or by letting $i \to M - i$ and $j \to M - j$,

$$\sum_{j=0}^{M} R_{j-i} G_j^{(M)} = R_{-(i+1)} \tag{2.8.14}$$

where

$$G_j^{(M)} \equiv \frac{x_{M-j}^{(M)} - x_{M-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.15}$$

To put this another way,

$$x_{M-j}^{(M+1)} = x_{M-j}^{(M)} - x_{M+1}^{(M+1)} G_j^{(M)} \qquad j = 0, \ldots, M \tag{2.8.16}$$

Thus, if we can use recursion to find the order $M$ quantities $x^{(M)}$ and $G^{(M)}$ *and* the single order $M + 1$ quantity $x_{M+1}^{(M+1)}$, then all of the other $x_j^{(M+1)}$'s will follow. Fortunately, the quantity $x_{M+1}^{(M+1)}$ follows from equation (2.8.12) with $i = M + 1$,

$$\sum_{j=0}^{M} R_{M+1-j} x_j^{(M+1)} + R_0 x_{M+1}^{(M+1)} = y_{M+1} \tag{2.8.17}$$

For the unknown order $M + 1$ quantities $x_j^{(M+1)}$ we can substitute the previous order quantities in $G$ since

$$G_{M-j}^{(M)} = \frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.18}$$

The result of this operation is

$$x_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} x_j^{(M)} - y_{M+1}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.19}$$

The only remaining problem is to develop a recursion relation for $G$. Before we do that, however, we should point out that there are actually two distinct sets of solutions to the original linear problem for a nonsymmetric matrix, namely right-hand solutions (which we have been discussing) and left-hand solutions $z_i$. The formalism for the left-hand solutions differs only in that we deal with the equations

$$\sum_{j=0}^{M} R_{j-i} z_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.20}$$

Then, the same sequence of operations on this set leads to

$$\sum_{j=0}^{M} R_{i-j} H_j^{(M)} = R_{i+1} \tag{2.8.21}$$

where

$$H_j^{(M)} \equiv \frac{z_{M-j}^{(M)} - z_{M-j}^{(M+1)}}{z_{M+1}^{(M+1)}} \tag{2.8.22}$$

(compare with 2.8.14 – 2.8.15). The reason for mentioning the left-hand solutions now is that, by equation (2.8.21), the $H_j$'s satisfy exactly the same equation as the $x_j$'s except for the substitution $y_i \to R_{i+1}$ on the right-hand side. Therefore we can quickly deduce from equation (2.8.19) that

$$H_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} H_j^{(M)} - R_{M+2}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.23}$$

By the same token, $G$ satisfies the same equation as $z$, except for the substitution $y_i \to R_{-(i+1)}$. This gives

$$G_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{j-M-1} G_j^{(M)} - R_{-M-2}}{\sum_{j=0}^{M} R_{j-M-1} H_{M-j}^{(M)} - R_0} \tag{2.8.24}$$

The same "morphism" also turns equation (2.8.16), and its partner for $z$, into the final equations

$$\begin{aligned} G_j^{(M+1)} &= G_j^{(M)} - G_{M+1}^{(M+1)} H_{M-j}^{(M)} \\ H_j^{(M+1)} &= H_j^{(M)} - H_{M+1}^{(M+1)} G_{M-j}^{(M)} \end{aligned} \tag{2.8.25}$$

Now, starting with the initial values

$$x_0^{(0)} = y_0/R_0 \qquad G_0^{(0)} = R_{-1}/R_0 \qquad H_0^{(0)} = R_1/R_0 \tag{2.8.26}$$

we can recurse away. At each stage $M$ we use equations (2.8.23) and (2.8.24) to find $H_{M+1}^{(M+1)}$, $G_{M+1}^{(M+1)}$, and then equation (2.8.25) to find the other components of $H^{(M+1)}, G^{(M+1)}$. From there the vectors $x^{(M+1)}$ and/or $z^{(M+1)}$ are easily calculated.

The program below does this. It incorporates the second equation in (2.8.25) in the form

$$H_{M-j}^{(M+1)} = H_{M-j}^{(M)} - H_{M+1}^{(M+1)} G_j^{(M)} \tag{2.8.27}$$

so that the computation can be done "in place."

Notice that the above algorithm fails if $R_0 = 0$. In fact, because the bordering method does not allow pivoting, the algorithm will fail if any of the diagonal principal minors of the original Toeplitz matrix vanish. (Compare with discussion of the tridiagonal algorithm in §2.4.) If the algorithm fails, your matrix is not necessarily singular — you might just have to solve your problem by a slower and more general algorithm such as $LU$ decomposition with pivoting.

The routine that implements equations (2.8.23) – (2.8.27) is also due to Rybicki. Note that the routine's `r[n-1+j]` is equal to $R_j$ above, so that subscripts on the `r` array vary from 0 to $2N - 2$.

toeplz.h

```
void toeplz(VecDoub_I &r, VecDoub_O &x, VecDoub_I &y)
```

Solves the Toeplitz system $\sum_{j=0}^{N-1} R_{(N-1+i-j)} x_j = y_i$ $(i = 0, \ldots, N-1)$. The Toeplitz matrix need not be symmetric. `y[0..n-1]` and `r[0..2*n-2]` are input arrays; `x[0..n-1]` is the output array.

```
{
    Int j,k,m,m1,m2,n1,n=y.size();
    Doub pp,pt1,pt2,qq,qt1,qt2,sd,sgd,sgn,shn,sxn;
    n1=n-1;
```

```
    if (r[n1] == 0.0) throw("toeplz-1 singular principal minor");
    x[0]=y[0]/r[n1];                    Initialize for the recursion.
    if (n1 == 0) return;
    VecDoub g(n1),h(n1);
    g[0]=r[n1-1]/r[n1];
    h[0]=r[n1+1]/r[n1];
    for (m=0;m<n;m++) {                 Main loop over the recursion.
        m1=m+1;
        sxn = -y[m1];                   Compute numerator and denominator for x from eq.
        sd = -r[n1];                        (2.8.19),
        for (j=0;j<m+1;j++) {
            sxn += r[n1+m1-j]*x[j];
            sd += r[n1+m1-j]*g[m-j];
        }
        if (sd == 0.0) throw("toeplz-2 singular principal minor");
        x[m1]=sxn/sd;                   whence x.
        for (j=0;j<m+1;j++)             Eq. (2.8.16).
            x[j] -= x[m1]*g[m-j];
        if (m1 == n1) return;
        sgn = -r[n1-m1-1];              Compute numerator and denominator for G and H,
        shn = -r[n1+m1+1];                  eqs. (2.8.24) and (2.8.23),
        sgd = -r[n1];
        for (j=0;j<m+1;j++) {
            sgn += r[n1+j-m1]*g[j];
            shn += r[n1+m1-j]*h[j];
            sgd += r[n1+j-m1]*h[m-j];
        }
        if (sgd == 0.0) throw("toeplz-3 singular principal minor");
        g[m1]=sgn/sgd;                  whence G and H.
        h[m1]=shn/sd;
        k=m;
        m2=(m+2) >> 1;
        pp=g[m1];
        qq=h[m1];
        for (j=0;j<m2;j++) {
            pt1=g[j];
            pt2=g[k];
            qt1=h[j];
            qt2=h[k];
            g[j]=pt1-pp*qt2;
            g[k]=pt2-pp*qt1;
            h[j]=qt1-qq*pt2;
            h[k--]=qt2-qq*pt1;
        }
    }                                   Back for another recurrence.
    throw("toeplz - should not arrive here!");
}
```

If you are in the business of solving *very* large Toeplitz systems, you should find out about so-called "new, fast" algorithms, which require only on the order of $N(\log N)^2$ operations, compared to $N^2$ for Levinson's method. These methods are too complicated to include here. Papers by Bunch [6] and de Hoog [7] will give entry to the literature.

## CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 5 [also treats some other special forms].

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), §19.[1]

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).[2]